

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards
(BL2,BL6)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks:25

Annexure A

Coding challenge

Code of Conduct:

*I (Y.Varun Kumar Reddy / 192424230) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Annexure A

Coding challenge

1. Write a Python script that generates a star topology diagram (ASCII or graphical) given n hosts and a central switch. The script must print each host label, the switch name, and the connecting Cat6 UTP cable length.
2. Code a function `validate_segment(length_m)` that returns True if the UTP segment is within IEEE 802.3 limits (≤ 100 m) and False with an error message otherwise. Include at least 3 test cases.
3. Write a script that simulates CSMA/CD on a star topology with 4 hosts. Log every collision detection event and the back-off interval to a file named `csma_log.txt`.
4. Using Python's `socket` library, write a client-server program that models two hosts communicating through a star switch. Demonstrate message forwarding by printing the switch's MAC address table after each frame transmission.
5. Create a class `StarSwitch` with methods `add_port(host_id)`, `remove_port(host_id)`, and `broadcast(frame)`. Broadcast must send a frame to all ports except the source and log each delivery.

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
Code Quality	15%	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done

```

n = int(input("Enter number of hosts: "))

print("\nSTAR TOPOLOGY")
print("Central Switch: SWITCH")

for i in range(1, n + 1):
    host = "Host" + str(i)
    length = int(input("Enter cable length for " + host + " (m): "))
    print(host, "----", length, "m Cat6 UTP ---- SWITCH")

```

Enter number of hosts: 4

```

STAR TOPOLOGY
Central Switch: SWITCH
Enter cable length for Host1 (m): 10
Host1 ---- 10 m Cat6 UTP ---- SWITCH
Enter cable length for Host2 (m): 20
Host2 ---- 20 m Cat6 UTP ---- SWITCH
Enter cable length for Host3 (m): 30
Host3 ---- 30 m Cat6 UTP ---- SWITCH
Enter cable length for Host4 (m): 40
Host4 ---- 40 m Cat6 UTP ---- SWITCH

```

```

def validate_segment(length_m):
    if length_m <= 100:
        return True
    else:
        return False

test1 = 50
test2 = 100
test3 = 120

print(test1, "m ->", validate_segment(test1))

print(test2, "m ->", validate_segment(test2))

print(test3, "m ->", validate_segment(test3))

if test3 > 100:
    print("Error: UTP segment cannot exceed 100 m")

```

```

50 m -> True
100 m -> True
120 m -> False
Error: UTP segment cannot exceed 100 m

```

```

import random

hosts = ["Host1", "Host2", "Host3", "Host4"]

log = open("csma_log.txt", "w")

for i in range(5):

    host1 = random.choice(hosts)
    host2 = random.choice(hosts)

    if host1 == host2:

        message = host1 + " transmitted successfully"

        print(message)
        log.write(message + "\n")

    else:

        backoff = random.randint(1, 5)

        message1 = "Collision detected: " + host1 + " and " + host2
        message2 = "Back-off interval: " + str(backoff)

        print(message1)
        print(message2)

```

```

log.write(message1 + "\n")
log.write(message2 + "\n")

log.close()

print("\nCSMA/CD simulation completed.")
print("Log saved as csma_log.txt")

```

```

Collision detected: Host2 and Host4
Back-off interval: 3
Collision detected: Host4 and Host3
Back-off interval: 1
Host1 transmitted successfully
Collision detected: Host4 and Host1
Back-off interval: 5
Collision detected: Host3 and Host1
Back-off interval: 4

CSMA/CD simulation completed.
Log saved as csma_log.txt

```

```

import socket
import threading
import time

HOST = "127.0.0.1"
PORT = 5000

mac_table = {
    "Host1": "AA:BB:CC:DD:01",
    "Host2": "AA:BB:CC:DD:02"
}

def server():

    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    server_socket.bind((HOST, PORT))
    server_socket.listen(1)

    print("Switch is waiting for Host1...")

    connection, address = server_socket.accept()

    message = connection.recv(1024).decode()

    print("\nSwitch received:", message)

    print("Switch forwards frame to Host2")

    print("\nSwitch MAC Address Table")

    for host, mac in mac_table.items():
        print(host, "->", mac)

    connection.send("Hello from Switch to Host2".encode())

    connection.close()
    server_socket.close()

def client():

    client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    client_socket.connect((HOST, PORT))

    message = "Hello from Host1"

    print("Host1 sends:", message)

    client_socket.send(message.encode())

    reply = client_socket.recv(1024).decode()

    print("Host2 receives:", reply)

```

```

    client_socket.close()
server_thread = threading.Thread(target=server)
server_thread.start()
time.sleep(1)
client()

```

Switch is waiting for Host1...
Host1 sends: Hello from Host1

Switch received: Hello from Host1
Switch forwards frame to Host2

Switch MAC Address Table
Host1 -> AA:BB:CC:DD:01
Host2 -> AA:BB:CC:DD:02
Host2 receives: Hello from Switch to Host2

```

class StarSwitch:

    def __init__(self):
        self.ports = []

    def add_port(self, host_id):
        self.ports.append(host_id)
        print(host_id, "connected")

    def remove_port(self, host_id):
        if host_id in self.ports:
            self.ports.remove(host_id)
            print(host_id, "removed")

    def broadcast(self, frame):

        source = frame["source"]
        message = frame["message"]

        print("\nBroadcasting:", message)

        for host in self.ports:

            if host != source:
                print("Frame delivered to", host)
switch = StarSwitch()

switch.add_port("Host1")
switch.add_port("Host2")
switch.add_port("Host3")
switch.add_port("Host4")

frame = {
    "source": "Host1",
    "message": "Hello Network"
}

switch.broadcast(frame)

switch.remove_port("Host4")

```

Host1 connected
Host2 connected
Host3 connected
Host4 connected

Broadcasting: Hello Network
Frame delivered to Host2
Frame delivered to Host3
Frame delivered to Host4
Host4 removed

